

Tarea

1. Lista de Códigos de Estado HTTP (Los "Errores" y Respuestas)

Cuando un cliente (como un navegador o una aplicación móvil) hace una petición a un servidor web, el servidor siempre responde con un **Código de Estado HTTP** de tres dígitos. Estos números te indican rápidamente qué pasó con la solicitud. Se dividen en cinco bloques principales:

- **1xx (Informativos):** La petición fue recibida y el proceso continúa. Rara vez los verás en el día a día.
- **2xx (Éxito):** Todo salió bien.
 - **200 OK:** Es el más común. Significa que la solicitud fue exitosa (por ejemplo, pediste datos y el servidor te los entregó).
 - **201 Created:** Se creó un recurso exitosamente (ej. guardaste un nuevo usuario en la base de datos).
- **3xx (Redirección):** El cliente debe tomar una acción adicional para completar la petición.
 - **301 Moved Permanently:** La página o recurso que buscas se movió a otra URL de forma definitiva.
- **4xx (Errores del Cliente):** El problema está de tu lado (escribiste mal la URL, enviaste datos incorrectos, o no tienes permisos).
 - **400 Bad Request:** El servidor no entendió la petición porque la sintaxis es inválida.
 - **401 Unauthorized:** Necesitas autenticarte (iniciar sesión) para acceder.
 - **403 Forbidden:** Estás autenticado, pero no tienes permiso para ver ese recurso.
 - **404 Not Found:** El clásico. El servidor no encontró lo que estabas buscando.
- **5xx (Errores del Servidor):** El servidor falló al intentar procesar una solicitud que era válida.
 - **500 Internal Server Error:** Un error genérico del lado del servidor (un fallo en el código, por ejemplo).
 - **502 Bad Gateway / 503 Service Unavailable:** El servidor está caído, sobrecargado o hay un problema de red interno.

2. ¿Cómo documentar con Swagger en Express?

Swagger (ahora conocido como OpenAPI) es una herramienta fantástica para crear documentación interactiva de tus APIs. En un proyecto de Node.js con Express, se hace a través de un par de librerías.

Paso 1: Instalar dependencias

En tu terminal, dentro de tu proyecto, instala estas dos librerías:

Bash

```
npm install swagger-ui-express swagger-jsdoc
```

Paso 2: Configurar Express

Aquí tienes un ejemplo básico de cómo se integra. Lo que haces es crear una configuración, pasarla a la librería, y exponer una ruta (usualmente /api-docs) donde se verá la interfaz gráfica.

JavaScript

```
const express = require('express');
const swaggerJsDoc = require('swagger-jsdoc');
const swaggerUi = require('swagger-ui-express');
const app = express();
// 1. Opciones de configuración de Swagger
const swaggerOptions = {
  swaggerDefinition: {
    openapi: '3.0.0', // Versión de OpenAPI
    info: {
      title: 'Mi API de Tarea',
      version: '1.0.0',
```

```

    description: 'Documentación de mi API en Express'
  }
},
// 2. ¿Dónde están los comentarios que Swagger debe leer?
apis: ['app.js'],
};
// 3. Generar la documentación
const swaggerDocs = swaggerJsDoc(swaggerOptions);
// 4. Crear la ruta para ver la interfaz gráfica
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerDocs));
// 5. Ejemplo de una ruta documentada (Nota el comentario especial @swagger)
/**
 * @swagger
 * /usuarios:
 * get:
 * summary: Obtiene una lista de usuarios
 * responses:
 * 200:
 * description: Lista obtenida con éxito (Aquí ves en acción el código 200)
 */
app.get('/usuarios', (req, res) => {
  res.status(200).json([
    { nombre: "Ana" },
    { nombre: "Juan" }
  ]);
});
app.listen(3000, () => console.log('Servidor en http://localhost:3000/api-docs'));

```

3. Patrones de Diseño (Basado en Refactoring.guru)

La página web *Refactoring.guru* es una de las mejores bibliotecas modernas para entender la arquitectura de software de forma visual y estructurada.

¿Qué es un patrón de diseño?

No es una pieza de código terminada que puedas copiar y pegar. Es una **solución general, conceptual y reutilizable** a un problema que ocurre habitualmente en el diseño de software. Piensa en ello como un plano arquitectónico: te da la idea de cómo resolver el problema, pero tú debes implementarlo en tu propio código. Al programar la lógica de una aplicación, ya sea armando el backend en Node o construyendo interfaces móviles complejas en lenguajes como Kotlin, aplicar estos patrones te ahorra reinventar la rueda.

¿Para qué sirven?

- **Velocidad:** Ofrecen paradigmas de desarrollo ya probados por miles de desarrolladores.
- **Comunicación:** Proporcionan un vocabulario común. Es más fácil decirle a tu equipo "usemos un patrón *Singleton* aquí" que explicar toda la lógica desde cero.
- **Claridad:** Hacen que el código sea más predecible y fácil de mantener.

La mejor manera de modelar y comprender cómo interactúan las piezas de un patrón antes de tirar una sola línea de código es a través de un **diagrama de clases**, ya que ahí puedes ilustrar exactamente qué objeto hereda o se compone de otro.

Tipos de Patrones:

Se dividen en tres grandes familias según su propósito:

1. **Patrones Creacionales (Creational):** Se ocupan de **cómo se crean los objetos**. Su objetivo es separar la lógica de creación del objeto del resto del sistema, haciéndolo más flexible.
 - *Ejemplo:* **Factory Method** (delega la creación de objetos a subclases) o **Singleton** (garantiza que solo exista una única instancia de una clase en todo el programa).
2. **Patrones Estructurales (Structural):**

Tratan sobre **cómo se componen los objetos y las clases** para formar estructuras más grandes y complejas. Si piensas en cómo se organizan los módulos de un sistema en un diagrama de componentes, estos patrones dictan cómo conectar esas piezas de forma eficiente sin que todo colapse si cambias una parte.

 - *Ejemplo:* **Adapter** (permite que clases con interfaces incompatibles trabajen juntas) o **Decorator** (añade funcionalidades a un objeto dinámicamente).
3. **Patrones de Comportamiento (Behavioral):**

Se centran en la **comunicación y la asignación de responsabilidades** entre los

objetos. Responden a la pregunta: ¿Cómo interactúan y se pasan mensajes mis clases?

- *Ejemplo:* **Observer** (un objeto notifica a otros sobre cambios en su estado, muy común en interfaces gráficas) o **Strategy** (permite intercambiar algoritmos en tiempo de ejecución).